Herramientas Computacionales para Economistas

Universidad Nacional de La Plata

Clase 4

Belén Michel Torino

Temario de la clase

1. Pandas (continuación de la clase 3)
2. Visualizaciones con seaborn (continuación de la clase 3)
3. Expresiones regulares
4. Widgets en Jupyter

Repaso de la clase anterior

En la clase pasada vimos:

1. **NumPy**: arrays, operaciones elemento a elemento y números aleatorios.
2. **pandas**: leer, explorar, filtrar, unir y agrupar datos tabulares.
3. **Matplotlib**: gráficos con `fig, ax`.

Tres ideas que hoy volvemos a usar:

- Un método devuelve un resultado nuevo: si lo queremos conservar, hay que asignarlo.
- Las operaciones entre columnas se aplican fila por fila, sin ciclos.
- `groupby` + `agg` resume: devuelve *una fila por grupo*.

Objetivos de la clase

Al terminar esta clase van a poder:

- **pandas**: transformar y filtrar datos *dentro de cada grupo*, y pasar una tabla de formato ancho a largo.
- **seaborn**: hacer gráficos estadísticos directamente desde un `DataFrame`, con poco código.
- **Expresiones regulares**: leer y escribir patrones para buscar, extraer y limpiar texto, y *comprobar* que hacen lo que queremos.
- ***Widgets***: agregar controles interactivos a un notebook de Jupyter.

Los tres primeros son de uso diario en una tesis empírica. El cuarto es una herramienta práctica para explorar datos.

Cómo aprovechar esta clase

1. **Notebook abierto.** El código de las slides está en la carpeta `Clase4` del repositorio. Ejecuten cada ejemplo.
2. **Pausas para predecir.** En las slides tituladas *Pausa*, detengan el video y escriban qué creen que va a devolver el código *antes* de ejecutarlo.
3. **Cuestionario.** Hay tres preguntas a lo largo de la clase. Respóndanlas sin ayuda: sirven para saber qué repasar.
4. **No memoricen la sintaxis.** Importa reconocer qué operación necesitan y saber verificar el resultado.

Si usan un asistente de IA mientras estudian

Pídanle una *explicación* o una *pista*, no la respuesta. En el cuestionario y en los trabajos prácticos valen las reglas de cada consigna.

Temario de la clase

1. Pandas (continuación de la clase 3)
2. Visualizaciones con seaborn (continuación de la clase 3)
3. Expresiones regulares
4. Widgets en Jupyter

Pandas (continuación de la clase 3)

En la clase 3 vimos cómo:

- crear un `DataFrame` desde cero y cargarlo desde un archivo;
- filtrar filas y columnas;
- apilar (`concat`) y unir (`merge`) `DataFrames`;
- agrupar con `groupby` y resumir con `agg` (cuenta, suma, promedio).

En esta parte cerramos pandas con tres herramientas nuevas:

1. **transformar** una columna dentro de cada grupo;
2. **filtrar** grupos enteros;
3. **reformatear** una tabla: de ancho a largo.

Tres cosas que se pueden hacer con un grupo

Después de `df.groupby("grupo")` hay tres familias de operaciones. La diferencia está en **cuántas filas devuelven**:

Método	**Devuelve**	**En Stata**
`agg`	una fila por grupo	`collapse (mean) x, by(g)`
`transform`	las mismas filas que la tabla original	`bysort g: egen m = mean(x)`
`filter`	solo las filas de los grupos que cumplen una condición	`bysort g: keep if _N >= 5`

Ejemplos de uso:

- `transform`: estandarizar un salario con la media y el desvío de su provincia; imputar faltantes con la media del grupo.
- `filter`: descartar los grupos con pocas observaciones.

Un `DataFrame` de ejemplo

Tres grupos de tamaño distinto. En cada uno, `x` es normal con media igual al valor de `media` y desvío 1.

```
import numpy as np
import pandas as pd

np.random.seed(73)      # semilla: todos obtenemos los mismos números
df = pd.DataFrame({"media": [0]*1000 + [1]*1000 + [2]*100})

def nueva_columna(tabla):
    return tabla["media"] + np.random.randn(tabla.shape[0])

df = df.assign(x=nueva_columna)   # assign devuelve un DataFrame nuevo
df
```

```
      media         x
0         0  0.576813
1         0  2.131109
...     ...       ...
2098      2  1.372851
2099      2  0.964190
[2100 rows x 2 columns]
```

Funciones anónimas: `lambda`

Cuando una función es tan corta que solo se usa una vez, podemos escribirla sin nombre:

```
lambda argumento: expresión_que_devuelve
```

Estas dos formas son equivalentes:

```python
# con una función con nombre
def nueva_columna(d):
    return d["media"] + np.random.randn(d.shape[0])

df = df.assign(x=nueva_columna)

# con una función anónima
df = df.assign(x=lambda d: d["media"] + np.random.randn(d.shape[0]))
```

- No lleva `def`, ni nombre, ni `return`: devuelve el valor de la expresión.
- Se pasa la función, no su resultado: `nueva_columna` va sin paréntesis.

Transformaciones sobre grupos: estandarizar

Ejemplo. Estandarizar `x` con la media y el desvío estándar *de su grupo*.

```
df["t"] = df.groupby("media")["x"].transform(
    lambda s: (s - s.mean()) / s.std())
```

La función recibe la columna `x` de *un* grupo (una `Series`) y devuelve otra del mismo largo.

Verificamos: dentro de cada grupo, `t` debe tener media 0 y desvío 1.

```
df.groupby("media")[["x", "t"]].agg(["mean", "std"]).round(2)
```

	x		t	
	mean	std	mean	std
media				
0	-0.01	0.97	0.0	1.0
1	1.00	0.99	-0.0	1.0
2	2.04	1.02	-0.0	1.0

`df` sigue teniendo 2100 filas: `transform` no cambia la cantidad de filas.

Transformaciones sobre grupos: imputar faltantes

Ejemplo. Reemplazar cada faltante por la media de su grupo.

```
sal = pd.DataFrame({
    "provincia": ["A", "A", "A", "B", "B", "B"],
    "salario":   [100, np.nan, 140, 200, 220, np.nan]})

sal["salario_imp"] = sal.groupby("provincia")["salario"].transform(
    lambda s: s.fillna(s.mean()))
sal
```

```
  provincia  salario  salario_imp
0         A    100.0        100.0
1         A      NaN        120.0
2         A    140.0        140.0
3         B    200.0        200.0
4         B    220.0        220.0
5         B      NaN        210.0
```

- En la fila 1 falta el salario: se completa con 120, la media de la provincia A. En la fila 5, con 210, la media de B.
- Guardamos el resultado en una columna nueva: el dato original no se pierde.
- Imputar es una decisión metodológica, no un trámite: hay que informarla.

Filtrado por grupos

`filter` recibe una función que se evalúa *una vez por grupo* y devuelve `True` o `False`.

Se conservan los grupos para los que la función devuelve `True`.

Ejemplo. Eliminar los grupos con menos de 500 observaciones (es decir, conservar los que tienen 500 o más):

```
1 grandes = df.groupby("media").filter(lambda g: len(g) >= 500)
2 grandes.shape
```

```
(2000, 3)
```

Con la condición opuesta nos quedamos con el grupo chico:

```
1 df.groupby("media").filter(lambda g: len(g) < 500).shape
```

```
(100, 3)
```

Error frecuente: escribir la condición de lo que se quiere eliminar. La condición describe lo que se conserva.

Pausa: ¿cuántas filas devuelve cada línea?

```
p = pd.DataFrame({"g": ["a", "a", "a", "b", "b"],
                  "v": [ 1,   2,   3,   10,  20]})

p.groupby("g")["v"].mean()                          # (1)
p.groupby("g")["v"].transform("mean")               # (2)
p.groupby("g").filter(lambda d: d["v"].mean() > 5)  # (3)
```

Pausa. Detengan el video y anoten su predicción

- ¿Cuántas filas tiene el resultado de (1), de (2) y de (3)?
- ¿Qué valores tiene el resultado de (2)?

Después ejecuten el código y comparen. Si alguna predicción falló, vuelvan a la tabla de la slide 8.

Reformatear un `DataFrame`

La forma habitual de organizar una tabla es poner las *observaciones* en las filas y las *variables* en las columnas.

Pero qué es una observación depende de lo que queramos hacer después:

- **Formato ancho**: una fila por provincia y una columna por año.
- **Formato largo**: una fila por provincia y año.

Los datos son los mismos; cambia la unidad de observación. La mayoría de las funciones de gráficos y de estimación esperan el formato largo.

pandas	**Stata**
`melt`: de ancho a largo	`reshape long`
`pivot`: de largo a ancho	`reshape wide`

De ancho a largo: `melt`

`id_vars` indica las columnas que identifican a cada fila. Las demás se apilan en dos columnas: `variable` y `value`.

`df3`

	first	last	height	weight
0	John	Doe	5.5	130
1	Mary	Bo	6.0	150



`df3.melt(id_vars=["first", "last"])`

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130.0
3	Mary	Bo	weight	150.0

- 2 personas $\times$ 2 columnas de medidas $=$ 4 filas.
- `value` mezcla alturas y pesos: una sola columna, un solo tipo de dato (por eso `130` pasa a `130.0`).

`melt` con un panel de datos

```
ancho = pd.DataFrame({"provincia": ["Chaco", "Salta"],
    "2022": [10.5, 12.0], "2023": [11.0, 12.4], "2024": [11.8, 13.1]})

largo = ancho.melt(id_vars="provincia",
                   var_name="anio", value_name="valor")
largo
```

```
  provincia  anio  valor
0     Chaco  2022   10.5
1     Salta  2022   12.0
2     Chaco  2023   11.0
3     Salta  2023   12.4
4     Chaco  2024   11.8
5     Salta  2024   13.1
```

- `var_name` y `value_name` dan nombre a las dos columnas nuevas.
- `anio` queda como texto: eran nombres de columnas. Para usarlo como número, `largo["anio"].astype(int)`.
- El camino inverso es `largo.pivot(index="provincia", columns="anio", values="valor")`.

Pregunta 1 (cuestionario)

Supongamos que tenemos el `DataFrame df`:

	Nombre	**Apellido**	**Edad**	**Dirección**	**Ciudad**
0	Mabel	García	40	San Martín 900	La Plata
1	Juan	Pérez	50	Belgrano 1770	Rosario
2	Héctor	Rossi	60	Sarmiento 1888	Santa Rosa

¿Qué dimensiones (`shape`) tiene el resultado de

```
df.melt(id_vars=["Nombre", "Apellido"])
```

?

Expresarlo como filas × columnas.

Primero respondan sin ejecutar nada. Después, comprueben la respuesta en el notebook.

Temario de la clase

1. Pandas (continuación de la clase 3)
2. Visualizaciones con seaborn (continuación de la clase 3)
3. Expresiones regulares
4. Widgets en Jupyter

Visualizaciones con `seaborn`

`seaborn` es un paquete de gráficos estadísticos construido sobre Matplotlib.

- Recibe un `DataFrame` y los *nombres* de las columnas.
- Se ocupa de los colores, las leyendas y los paneles.
- Como por debajo usa Matplotlib, lo que aprendieron en la clase 3 sigue sirviendo para ajustar títulos y ejes.

Se instala una sola vez, desde la terminal:

`conda install seaborn` o `pip install seaborn`

Y se importa al comienzo de cada notebook[1]:

```
1 import seaborn as sns
```

[1] El alias `sns` no es obligatorio, pero es el que usa toda la documentación.

Los datos del ejemplo: `tips`

`seaborn` trae conjuntos de datos de ejemplo. Usamos `tips`: propinas en un restaurante. *(La primera vez los descarga de internet.)*

```
propinas_df = sns.load_dataset("tips")
propinas_df.shape
```

```
(244, 7)
```

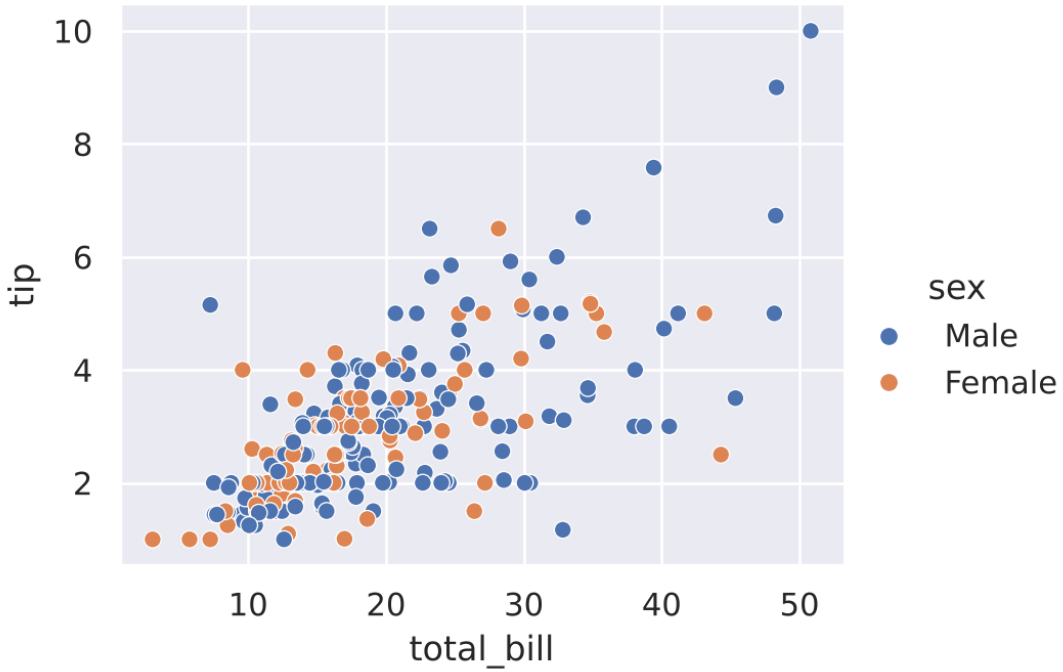
```
propinas_df.head(3)
```

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
```

Cada fila es una mesa: total de la cuenta, propina, sexo de quien pagó, si había fumadores, día, turno (almuerzo o cena) y cantidad de comensales. El resultado es un `DataFrame` de pandas.

Un gráfico de dispersión: `relplot`

```
sns.set_theme()                                    # tema por defecto de seaborn
sns.relplot(data=propinas_df, x="total_bill", y="tip", hue="sex")
```



- `data`: el `DataFrame`.
- `x`, `y`: nombres de columnas, como texto.
- `hue`: columna que define el color.

`relplot` viene de *relational plot*: muestra la relación entre dos variables.

Paneles: `row` y `col`

```
sns.relplot(data=propinas_df, x="total_bill", y="tip", hue="sex",
            row="time", col="smoker")
```



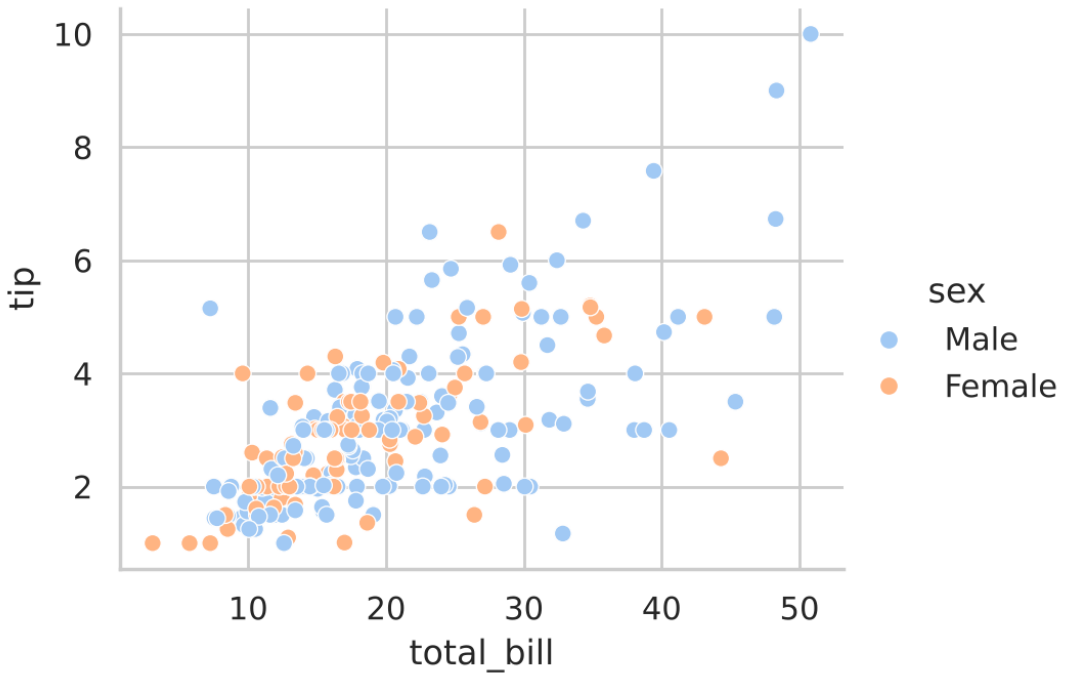
Dividir un gráfico en paneles según una variable se llama *facetar*.

- `row`: una fila de paneles por turno.
- `col`: una columna de paneles por condición de fumador.

Los ejes son compartidos: los paneles se pueden comparar directamente.

Tema, estilo y paleta de colores

```
sns.set_theme(style="whitegrid", palette="pastel")
sns.relplot(data=propinas_df, x="total_bill", y="tip", hue="sex")
```



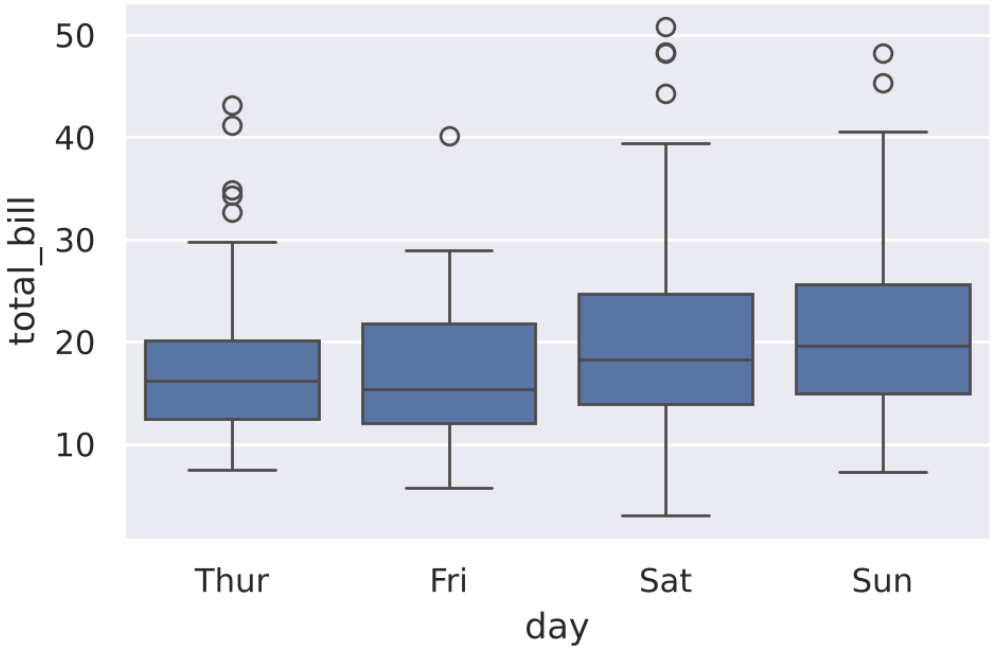
`set_theme` cambia el aspecto de *todos* los gráficos que hagamos después.

- `style`: fondo y grilla (`darkgrid` es el valor por defecto).
- `palette`: colores.
- `font_scale`: tamaño de la letra.

Para volver al tema inicial: `sns.set_theme()`.

Una variable categórica: `boxplot`

```
1 sns.boxplot(data=propinas_df, x="day", y="total_bill")
```



Un diagrama de caja muestra la distribución de `y` para cada categoría de `x`: mediana, cuartiles y valores atípicos.

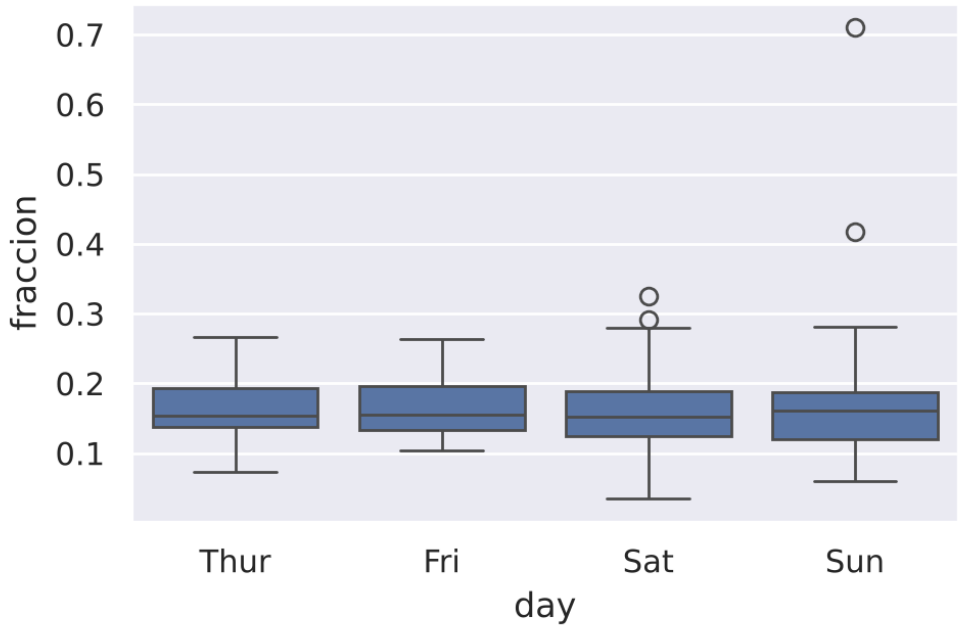
Las cuentas parecen más altas el fin de semana.

Cuidado: el gráfico sugiere una diferencia, pero no dice si es estadísticamente significativa.

Graficar una variable que todavía no existe

¿Y la propina como fracción de la cuenta? La columna no está en la tabla: la creamos con **assign** en el mismo paso.

```
sns.boxplot(
    data=propinas_df.assign(fraccion=lambda d: d["tip"] / d["total_bill"]),
    x="day", y="fraccion")
```



- **propinas_df** no se modifica: **assign** devuelve una tabla nueva, que solo se usa para el gráfico.
- La mediana ronda el 15 % todos los días: el fin de semana se gasta más, pero no se deja una fracción mayor.

Pregunta 2 (cuestionario)

Dado el `DataFrame` de propinas del ejemplo, `propinas_df`:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
...	...	...	...	...	...	...	...
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

¿Qué línea de código de `seaborn` escribirían para generar un *boxplot* que tenga en el eje horizontal el tamaño de la mesa y en el eje vertical el valor de la propina?

Escriban la línea sin ayuda y después ejecútenla: el gráfico les dice si está bien.

Temario de la clase

1. Pandas (continuación de la clase 3)
2. Visualizaciones con seaborn (continuación de la clase 3)
3. Expresiones regulares
4. Widgets en Jupyter

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un **patrón de texto**.

Se usan, por ejemplo, para:

- *buscar* y *buscar y reemplazar* en editores de texto y de código;
- validar lo que escribe un usuario (¿este texto tiene formato de e-mail?);
- reconocer las piezas de un lenguaje de programación (nombres, números, operadores);
- **minería de datos**: extraer información de texto no estructurado.

En el trabajo empírico:

- limpiar nombres de localidades, empresas o direcciones antes de un `merge`;
- extraer montos, fechas o números de norma de un documento;
- separar un campo mal cargado en sus componentes.

El módulo `re` y los *strings* crudos

En Python, las expresiones regulares se manejan con el módulo `re`. Viene con Python: no hay que instalarlo.

```
1 import re
```

Una costumbre para adoptar desde hoy: escribir los patrones como *strings* crudos, con una `r` antes de las comillas.

```
1 patron = r"\w+ente"       # bien: string crudo (raw string)
2 patron =  "\w+ente"       # funciona, pero Python muestra una advertencia
```

- En un *string* común, la barra invertida tiene significado propio: `"\n"` es un salto de línea.
- Con la `r`, Python deja las barras tal como están y se las pasa intactas al módulo `re`.
- Sin la `r`, las versiones actuales de Python emiten un `SyntaxWarning: invalid escape sequence`.

Un primer patrón: `compile` y `search`

```
1 frase = "¿Dónde va Vicente? Donde va la gente."
2
3 regex = re.compile(r"ente")      # objeto de tipo re.Pattern
4 regex.search(frase)              # busca la primera coincidencia
```

```
<re.Match object; span=(13, 17), match='ente'>
```

- `re.compile` convierte el *string* en un objeto `re.Pattern`, que tiene métodos para buscar.
- `search` devuelve un objeto `re.Match` con la **primera** coincidencia, o `None` si no hay ninguna.
- `span=(13, 17)`: la coincidencia empieza en la posición 13 y termina *antes* de la 17, como en un *slice*: `frase[13:17]` es `'ente'`.
- Es el `ente` de *Vicente*. El de *gente* viene después y `search` no lo informa.

Todas las coincidencias: `findall` y `finditer`

```
1 regex.findall(frase)                # lista de strings
```

```
['ente', 'ente']
```

```
1 for coincidencia in regex.finditer(frase):      # iterador de re.Match
2     print(coincidencia)
```

```
<re.Match object; span=(13, 17), match='ente'>
<re.Match object; span=(32, 36), match='ente'>
```

- `findall`: qué texto coincidió, pero no dónde.
- `finditer`: un objeto `re.Match` por coincidencia, con su posición. Devuelve un iterador: para ver sus elementos hay que recorrerlo con un `for` o convertirlo con `list()`.
- De un `re.Match` se obtiene el texto con `.group()` y las posiciones con `.start()` y `.end()`.

Conjuntos de caracteres

Los corchetes indican que en **una** posición puede ir **cualquiera** de los caracteres listados: `[caracteres]`.

Si el primer carácter es `^`, el conjunto se niega: `[^caracteres]` es cualquier carácter que **no** esté en la lista.

Patrón	**Coincide con**	**No coincide con**
`[Hhbcs]ola`	`Hola`, `hola`, `bola`, `cola`, `sola`	`mola`
`[^bc]ola`	`Hola`, `hola`, `sola`, `mola`	`bola`, `cola`

En la frase del ejemplo:

```
1 re.findall(r"Donde", frase)          # ['Donde']
2 re.findall(r"D[oó]nde", frase)       # ['Dónde', 'Donde']
3 re.findall(r"[^c]ente", frase)       # ['gente']
```

Las expresiones regulares distinguen mayúsculas de minúsculas y letras con tilde de letras sin tilde.

Conjuntos abreviados y predefinidos

Dentro de los corchetes se pueden escribir rangos:

`[a-z]`	una letra minúscula (sin tildes ni ñ)
`[A-Z]`	una letra mayúscula (sin tildes ni Ñ)
`[a-zA-Z]`	una letra
`[0-9]`	un dígito

Y hay conjuntos predefinidos, que se escriben sin corchetes:

`\d`	un dígito
`\w`	un carácter de palabra: letra (con o sin tilde), dígito o guion bajo
`\s`	un espacio en blanco: espacio, tabulación o salto de línea
`\D`, `\W`, `\S`	las negaciones de los anteriores

```
1 re.findall(r"\w+", "tasa_2024: 3,5%")      # ['tasa_2024', '3', '5']
```

Caracteres especiales

Algunos caracteres tienen un significado especial dentro de una expresión regular: no se representan a sí mismos.

. * ? + ^ $ \ | ( ) [ ] { }

No hace falta memorizarlos hoy: alcanza con reconocerlos cuando aparecen. En las próximas slides vemos qué hace cada uno.

¿Y si queremos buscar literalmente uno de esos caracteres? Lo precedemos con una barra invertida. Esto se llama *escapar* el carácter.

```
1 re.findall(r"1.5",  "1.5 y 105")     # ['1.5', '105']  el punto es comodín
2 re.findall(r"1\.5", "1.5 y 105")     # ['1.5']         punto literal
```

Anclas, comodín y cuantificadores

`^`	comienzo del *string* (fuera de los corchetes)
`$`	final del *string*
`.`	comodín: cualquier carácter, salvo el salto de línea
`*`	el elemento anterior, 0 o más veces
`+`	el elemento anterior, 1 o más veces
`?`	el elemento anterior, 0 o 1 vez
`{n}`, `{n,m}`	exactamente n veces; entre n y m veces

- Un cuantificador afecta al elemento que tiene **inmediatamente antes**: un carácter, un conjunto o un grupo entre paréntesis.
- `.*` coincide con cualquier secuencia de caracteres, incluso vacía, dentro de una misma línea.
- `\d{4}` coincide con cuatro dígitos seguidos, como un año.

Cuantificadores: recuperar las palabras completas

Con `ente` sabíamos que había dos coincidencias, pero no de qué palabras venían. Agregamos caracteres de palabra por delante:

```
frase2 = frase + " ¿Por qué? Porque es un ente."

re.findall(r"\w+ente", frase2)      # una o más letras antes
```

```
['Vicente', 'gente']
```

```
re.findall(r"\w*ente", frase2)      # cero o más letras antes
```

```
['Vicente', 'gente', 'ente']
```

- Con `+` se pierde la palabra *ente*: exige al menos un carácter antes.
- Con `*` aparece: admite que no haya ninguno.
- El patrón no exige que la palabra *termine* en *ente*: en *atentamente* hay coincidencia.

Alternativas: la barra vertical

La barra `|` entre dos expresiones significa **una u otra**: es un *o* lógico.

```
1 re.findall(r"\w*ente|D[oó]nde", frase2)
```

```
['Dónde', 'Vicente', 'Donde', 'gente', 'ente']
```

- Las coincidencias aparecen en el orden en que están en el texto, no en el orden de las alternativas.
- La barra separa **todo** lo que tiene a cada lado. Para limitar su alcance se usan paréntesis: `\.(csv|xlsx)`.

Pausa: ¿qué devuelve cada línea?

```
texto = "En 2023 la inflación fue 211,4% y en 2024, 117,8%"

re.findall(r"\d+", texto)              # (1)
re.findall(r"\d+,\d+%", texto)         # (2)
```

Pausa. Detengan el video y escriban las dos listas

- ¿Cuántos elementos tiene la lista (1)? ¿Cuáles son?
- ¿Y la lista (2)?
- Si quisieran quedarse solo con los años, ¿qué patrón usarían?

Después ejecútenlo y comparen. Lean el patrón símbolo por símbolo: `\d+` es «uno o más dígitos»; la coma y el `%` son caracteres literales.

Funciones y métodos del módulo `re`

En lo que sigue:

- `patron` es un *string* que representa una expresión regular;
- `regex` es el objeto `re.Pattern` que devuelve `re.compile(patron)`;
- `texto` es el *string* donde buscamos.

Cada operación se puede escribir de dos maneras equivalentes:

```
1 re.findall(patron, texto)          # función del módulo
2
3 regex = re.compile(patron)         # método del patrón compilado
4 regex.findall(texto)
```

- La primera es más corta. La segunda conviene cuando el mismo patrón se usa muchas veces.
- En las funciones del módulo, el patrón va **primero** y el texto **después**.

Buscar: `search`, `match` y `fullmatch`

Las tres devuelven un objeto `re.Match`, o `None` si no hay coincidencia. Cambia **dónde** tiene que estar:

		Patrón `\d+` sobre el texto...		
Función	**La coincidencia...**	`2024`	`2024 pesos`	`año 2024`
`re.search`	puede estar en cualquier lugar	sí	sí	sí
`re.match`	debe empezar al comienzo	sí	sí	no
`re.fullmatch`	debe abarcar todo el texto	sí	no	no

- `match` **no** exige que coincida el texto completo: solo que la coincidencia arranque en la primera posición.
- Para validar un dato entero (un código postal, una fecha, un CUIT) se usa `fullmatch`.
- Como `None` equivale a falso, se puede escribir `if re.search(patron, texto):`

Listar, dividir y reemplazar

Las otras cuatro operaciones:

`re.findall(patron, texto)`	lista con todas las coincidencias
`re.finditer(patron, texto)`	iterador de objetos `re.Match`
`re.split(patron, texto)`	lista con los pedazos que quedan al cortar `texto` en cada coincidencia
`re.sub(patron, repl, texto)`	*string* nuevo, con cada coincidencia reemplazada por `repl`

```
1 re.split(r"\s*[;,]\s*", "soja; maíz , trigo,girasol")
```

```
['soja', 'maíz', 'trigo', 'girasol']
```

```
1 re.sub(r"\s+", " ", "Av.   7   N°  776")
```

```
'Av. 7 N° 776'
```

Orden de `sub`: patrón, reemplazo, texto. Con el patrón compilado: `regex.sub(repl, texto)`.

Extraer partes de una coincidencia: grupos

Los paréntesis definen **grupos de captura**: permiten recuperar por separado la porción de texto que coincidió con cada parte del patrón.

```
regex = re.compile(r"(.*)\.(txt|csv|xls|doc)")
m = regex.match("archivo_imaginario.xls")

m.group(1)      # 'archivo_imaginario'
m.group(2)      # 'xls'
m.group(0)      # 'archivo_imaginario.xls'  (la coincidencia completa)
m.groups()      # ('archivo_imaginario', 'xls')
```

- Los grupos se numeran desde 1, de izquierda a derecha. El 0 es toda la coincidencia.
- El punto literal, `\.`, está fuera de los paréntesis: no queda en ningún grupo.
- Antes de llamar a `.group()` hay que comprobar que hubo coincidencia: si `m` es `None`, da error.

¿El patrón hace lo que queremos? Hay que probarlo

Un patrón puede funcionar con el ejemplo que teníamos en mente y fallar con otros. Probamos el de la slide anterior con más casos:

Texto	`regex.match`	`regex.fullmatch`
`datos.csv`	`datos.csv`	`datos.csv`
`tabla.v2.xls`	`tabla.v2.xls`	`tabla.v2.xls`
`foto.png`	`None`	`None`
`informe.xlsx`	`informe.xls`	`None`
`notas.docx`	`notas.doc`	`None`

Con `match`, un archivo `.xlsx` queda registrado como `.xls`, sin ningún aviso.

Cómo probar un patrón:

1. Casos que **deben** coincidir, incluidos los raros.
2. Casos que **no deben** coincidir: son los que revelan los errores.
3. En datos reales: contar cuántos coinciden y **mirar los que quedaron afuera**.

Los paréntesis cambian lo que devuelve `findall`

Queremos todos los porcentajes, con o sin decimales:

```
1 texto = "En 2023 la inflación fue 211,4% y en 2024, 117,8%"
2 re.findall(r"\d+(,\d+)?%", texto)
```

```
[',4', ',8']
```

Si el patrón tiene un grupo de captura, `findall` devuelve el contenido **del grupo**, no la coincidencia completa.

Para agrupar sin capturar se escribe `(?:...)`:

```
1 re.findall(r"\d+(?:,\d+)?%", texto)
```

```
['211,4%', '117,8%']
```

El primer patrón «se ve» correcto. La única forma de descubrir el problema es ejecutarlo y mirar la salida.

Expresiones regulares e inteligencia artificial

Un patrón es fácil de pedir y difícil de leer: por eso acá importa tanto saber verificar.

Usos que ayudan a aprender

- Pedir que explique un patrón, símbolo por símbolo, y contrastar la explicación con lo que vimos.
- Pedir una pista cuando están trabados, después de intentarlo.
- Pedir casos de prueba difíciles para un patrón que escribieron ustedes.

Uso que no ayuda

- Copiar un patrón que no pueden explicar. Si falla en la fila 3.000 de su base, no van a saber por qué.

Regla práctica

Venga de donde venga el patrón (de ustedes, de un foro o de un asistente), no se usa hasta pasar las tres pruebas de la slide 44.

Evidencia: Bastani et al. (*PNAS*, 2025); Shen y Tamkin (2026).

Aplicación: darle estructura al *Martín Fierro*

Un texto largo es un único *string*. Queremos convertirlo en una estructura que se pueda recorrer.

```
I - Cantor y Gaucho.

1
Aquí me pongo a cantar
Al compás de la vigüela,
Que el hombre que lo desvela
Una pena estraordinaria
Como la ave solitaria
Con el cantar se consuela.

2
Pido a los Santos del Cielo
...
```

Formato del archivo

- Cada parte empieza con un número romano, un guion y un título.
- Cada estrofa empieza con un número en una línea propia.
- Cada verso ocupa una línea.

Objetivo: una lista anidada.

- `versos[i]`: parte i
- `versos[i][j]`: estrofa j
- `versos[i][j][k]`: verso k

Archivo: `Clase4/archivos/martin_fierro.txt`, descargado de Project Gutenberg.

Paso 1: leer el archivo y separar las partes

```
with open("archivos/martin_fierro.txt", encoding="utf8") as archivo:
    martin_fierro = archivo.read()

partes = re.split(r"[IVX]+ - .*\n", martin_fierro)
len(partes)
```

```
13
```

`[IVX]+`	uno o más caracteres de números romanos
` - `	espacio, guion, espacio
`.*\n`	el título: cualquier cosa hasta el salto de línea

Hay 12 títulos, así que `split` devuelve 13 pedazos. El primero es lo que está antes de la parte I: el encabezado y una carta del autor. Lo quitamos:

```
encabezado = partes.pop(0)      # pop quita el elemento y lo devuelve
len(partes)                     # 12
```

Pasos 2 y 3: estrofas y versos

```
estrofas = []
for parte in partes:
    lista = re.split(r"\n\d+\n", parte)    # salto, número, salto
    lista.pop(0)                           # string vacío inicial
    estrofas.append(lista)

versos = []
for parte in estrofas:
    lista = []
    for estrofa in parte:
        lista.append(estrofa.strip().split("\n"))
    versos.append(lista)

versos[0][0]
```

```
['Aquí me pongo a cantar', 'Al compás de la vigüela,',
 'Que el hombre que lo desvela', 'Una pena estraordinaria',
 'Como la ave solitaria', 'Con el cantar se consuela.']
```

Para separar los versos no hace falta una expresión regular: el separador es siempre el mismo, y alcanza con el método `split` de los *strings*.

Paso 4: verificar el resultado

¿Cuántos versos tiene cada estrofa?

```
largos = []
for parte in versos:
    for estrofa in parte:
        largos.append(len(estrofa))

pd.Series(largos).value_counts().sort_index()
```

```
4       33
5        1
6      354
8        4
9        1
10       1
373      1
```

Una «estrofa» de 373 versos es una señal de alarma. Es la última: después del poema, el archivo trae la licencia de Project Gutenberg, y quedó pegada a la estrofa final.

Cortamos el texto antes del pie y repetimos los pasos 1 a 3:

```
texto = martin_fierro.split("End of Project Gutenberg")[0]
```

Resultado: 12 partes, 395 estrofas y 2318 versos; 355 estrofas tienen seis versos.

Buscar en el texto estructurado

Primera y última palabra de cada verso de la primera estrofa:

```
for verso in versos[0][0]:
    print(re.findall(r"^\w+|\w+$", verso))
```

```
['Aquí', 'cantar']
['Al']
['Que', 'desvela']
['Una', 'estraordinaria']
['Como', 'solitaria']
['Con']
```

En dos versos falta la última palabra: terminan en coma o en punto, y `\w+$` exige que la palabra llegue hasta el final.

Admitimos un signo de puntuación opcional:

```
re.findall(r"^\w+|\w+[.,;:]?$", verso)      # ['Al', 'vigüela,']
```

- En todo el poema, el primer patrón encuentra las dos palabras en el 39 % de los versos; el segundo, en el 93 %.
- El signo queda dentro de la coincidencia: `'vigüela,'`.
- ¿Qué pasa con el 7 % restante? Hay que mirarlo: queda para la clase sincrónica.

Pregunta 3 (cuestionario)

a. Escribir una expresión regular que coincida con un *string* que *contiene* una letra `a` seguida de cualesquiera caracteres, y termina en una letra `b`.

b. Escribir una expresión regular que coincida con un *string* que *contiene* una o más letras `a` seguidas inmediatamente de cero o más dígitos.

Antes de responder, pruébenlas

Para cada expresión, armen una lista con al menos tres textos que deben coincidir y tres que no, y comprueben el resultado con `re.search`:

```
1 casos = ["...", "...", "..."]          # completar
2 for caso in casos:
3     print(caso, bool(re.search(r"...", caso)))
```

«Contiene» quiere decir que la `a` no tiene por qué ser el primer carácter.

Temario de la clase

1. Pandas (continuación de la clase 3)
2. Visualizaciones con seaborn (continuación de la clase 3)
3. Expresiones regulares
4. Widgets en Jupyter

Widgets

Los *widgets* son controles interactivos (barras deslizantes, menús, cuadros de texto) que se muestran dentro de un notebook y permiten elegir valores sin modificar el código.

En Jupyter se usan con el módulo `ipywidgets`. Se instala una sola vez, desde la terminal:

```
conda install -c conda-forge ipywidgets
```
o bien `pip install ipywidgets`

Y se importa así:

```
1 import ipywidgets as widgets
```

El código de esta sección tiene que ejecutarse en un notebook (Jupyter, VS Code o Colab). En una consola de texto el control no se dibuja.

Crear un widget y leer su valor

```
w = widgets.IntSlider(min=0, max=10)
display(w)
```



El valor actual se consulta con el atributo `value`:

```
w.value          # 4
```

- `IntSlider` es una barra para elegir un número entero. `Textarea` es un cuadro de texto; `Dropdown`, un menú desplegable. Todos tienen `value`.
- Si movemos la barra, la celda que ya mostró `w.value` **no** se actualiza sola: hay que volver a ejecutarla.

Reactividad: `interact`

Para que el código se vuelva a ejecutar cada vez que cambia el widget usamos la función `interact`:

```
from ipywidgets import interact

def f(x):
    print("El doble de", x, "es", 2 * x)

interact(f, x=widgets.IntSlider(min=0, max=10));
```

- El primer argumento es una función definida por nosotros. Se pasa sin paréntesis.
- Los demás argumentos tienen el nombre de los argumentos de esa función, y como valor, el widget que los controla.
- Cada vez que movemos la barra, `interact` llama a `f` con el valor nuevo y actualiza la salida.
- El punto y coma final evita que el notebook imprima, además, la representación del objeto que devuelve `interact`.

Un gráfico interactivo

Elegimos, con un menú desplegable, la columna que define el color de los puntos:

```
def graficar(columna):
    sns.relplot(data=propinas_df, x="total_bill", y="tip", hue=columna)

interact(graficar,
         columna=widgets.Dropdown(options=["smoker", "sex", "time"]));
```

- La función recibe el nombre de una columna y dibuja el gráfico.
- `Dropdown` ofrece tres nombres de columnas.
- Al cambiar la opción, el gráfico se vuelve a dibujar.

Para qué sirve: explorar una base sin reescribir el código en cada intento. Por ejemplo, elegir la provincia, el año o la variable que se grafica.

En la clase sincrónica vamos a combinar widgets con expresiones regulares.

Resumen de la clase

- **pandas.** `agg` resume, `transform` conserva las filas, `filter` conserva grupos enteros. `melt` pasa de ancho a largo.
- **seaborn.** Se le pasa el `DataFrame` y los nombres de las columnas: `x`, `y`, `hue`, `row`, `col`.
- **Expresiones regulares.** Conjuntos, cuantificadores, anclas, alternativas y grupos. `search`, `match` y `fullmatch` no son lo mismo.
- **Widgets.** `interact` conecta un control con una función.

Una idea común a todo: el código que corre sin errores no necesariamente hace lo que queremos. Hay que mirar el resultado.

Antes de la clase sincrónica:

1. Ejecuten el notebook y hagan las dos pausas.
2. Respondan las tres preguntas del cuestionario.
3. Anoten sus dudas.

¡Fin de la clase 4!